Progressive Entropic Optimal Transport Solvers

Alexandre Lagier
ENS de Lyon, Computer Science Department

March 22, 2026

Abstract

Entropic optimal transport (EOT) has become the dominant approach for computing transport plans and maps in machine learning, but its performance is highly sensitive to the regularization parameter ε. We propose ProgOT, a novel family of progressive EOT solvers that decompose the transport problem into a sequence of simpler subproblems along a McCann-type interpolation path, using Sinkhorn's algorithm as a subroutine with dynamically scheduled parameters. We show that ProgOT yields statistically consistent map estimators and demonstrate through extensive experiments on synthetic and real single-cell datasets that it outperforms standard EOT methods.

1 Introduction

Optimal transport (OT) provides powerful tools for aligning datasets in machine learning applications. Given samples from source and target distributions, practitioners typically seek either a coupling matrix or a transport map. Entropic regularization has emerged as the standard approach, enabling efficient computation via Sinkhorn's algorithm.

However, this family of algorithm suffers from a critical weakness: performance depends heavily on the regularization parameter ε. Choosing ε poorly leads to biased maps and uninformative couplings.

We present ProgOT [1], which divides the transport problem into multiple steps along an interpolation path. Each step solves a simpler subproblem using Sinkhorn with scheduled parameters. This progressive approach proves more robust to hyperparameter choices while maintaining computational efficiency and statistical consistency.

We measure ProgOT's performances across extensive experiments on both synthetic and real-world datasets. We show that ProgOT partially solves hyperparameters tuning while yielding less diversified outputs.

2 Background

Optimal Transport Theory

Given a space $\mathcal{X} \subseteq \mathbb{R}^d$, let's denote $\mathcal{P}(\mathcal{X})$ the family of probability distributions on $\mathcal{X}$. Let's also take $c : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ a cost function on that same space. We consider here a translation invariant cost function $c(\mathbf{x}, \mathbf{y}) := h(\mathbf{x} - \mathbf{y})$. Given two probability distributions μ, ν, let's denote $\Gamma(\mu, \nu)$ as the set of probability distribution having left-marginal μ and right-marginal ν. Kantorovich problem which aim at defining a distance between two probability distributions is defined as follows,

$$\inf_{\pi \in \Gamma(\mu,\nu)} \int \int_{\mathcal{X} \times \mathcal{X}} h(\mathbf{x} - \mathbf{y}) d\pi \tag{1}$$

In Kantorovich problem, we try to find the best plan π coupling masses of μ to the closest masses of ν (if h quantized a distance). The minimizer π_0 of (1) is called an OT plan or OT coupling: there always

exists one. A more specific problem would be to map each mass of μ to another mass of ν: more formally, to find the optimal pushforward map T from μ to ν :

$$\inf_{T:T_{\#}\mu=\nu} \int_{\mathcal{X}} h(\mathbf{x} - T(\mathbf{x}))d\mu \tag{2}$$

The minimizer T_0 of (1) is called an OT map. Here, the existence of such optimal map is not guaranteed.

As it appears that (1) is a linear problem, we recall the dual of the Kantorovich problem also known as Brenier's Theorem [2]. Let's denote $\mathcal{C} = \{\mathbf{f}, \mathbf{g} | \mathbf{f}, \mathbf{g} : \mathcal{X} \to \mathbb{R}, \mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{y}) \leq c(\mathbf{x}, \mathbf{y})\}$, the dual formulation of Kantorovich problem is stated as,

$$\max_{\mathbf{f},\mathbf{g}\in\mathcal{C}} \int_{\mathcal{X}} \mathbf{f} d\mu + \int_{\mathcal{X}} \mathbf{g} d\nu \tag{3}$$

f, g are called potentials and the minimizers $(\mathbf{f}_0, \mathbf{g}_0)$ are called the OT potentials. As for many linear problems, dual problems often offer more flexibility for constrainted optimizing algorithms as some constraints are more difficult to optimize with than others.

Entropic Optimal Transport

We now recall the Entropic Optimal Transport problem. Given $\mathbf{X} = \mathbf{x}_1, \ldots, \mathbf{x}_n \in \mathbb{R}^{n\times d}$ and $\mathbf{Y} = \mathbf{y}_1, \cdots, \mathbf{y}_m \in \mathbb{R}^{m\times d}$ two populations with probability distributions $\mathbf{a} \in \mathbb{R}^n$ and $\mathbf{b} \in \mathbb{R}^m$. A finite formulation fo the dual Kantorovich problem can be written for the empirical probability distributions $\hat{\mu} = \sum_i \mathbf{a}_i \delta_{\mathbf{x}_i}$ and $\hat{\nu} = \sum_j \mathbf{b}_j \delta_{\mathbf{y}_j}$:

$$\max_{\mathbf{f},\mathbf{g}} \langle \mathbf{f}, \mathbf{a}\rangle + \langle \mathbf{g}, \mathbf{b}\rangle - \varepsilon \langle e^{\mathbf{f}/\varepsilon}, \mathbf{K} e^{\mathbf{g}/\varepsilon}\rangle \tag{4}$$

where $\varepsilon > 0$ is the regularization parameter and $\mathbf{K} = [\exp(-h(\mathbf{x}_i - \mathbf{y}_j)/\varepsilon)]_{i,j}$ is the Gibbs kernel of the cost matrix $\mathbf{C}$. This formulation yields more efficient optimizing algorithms as $\mathbf{f}, \mathbf{g}$ are now unconstrained. In Algorithm Figure 1, we present the (log-domain) Sinkhorn algorithm where we denote $\mathbf{f} \oplus \mathbf{g} = [\mathbf{f}_i + \mathbf{g}_j]_{i,j}$ and where $\min_\varepsilon(\mathbf{S}) := [-\varepsilon \log \left(\mathbf{1}^\top e^{-\mathbf{S}_{i,\cdot}/\varepsilon}\right)]_i$.

However, solving (4) is not equivalent to solving (3) : the minimizers $(\mathbf{f}_\varepsilon, \mathbf{g}_\varepsilon)$ are only approximations of the OT potentials $(\mathbf{f}_0, \mathbf{g}_0)$.[1]

From those potentials approximations $(\mathbf{f}^*_\varepsilon, \mathbf{g}^*_\varepsilon)$, one can build the entropic plan π_ε approximating the optimal plan π_0 with :

$$\pi_\varepsilon := [\exp((\mathbf{f}^*_{\varepsilon,i} + \mathbf{g}^*_{\varepsilon,j} - h(\mathbf{x}_i - \mathbf{y}_j))/\varepsilon)]_{i,j} \tag{5}$$

One can even build an empirical entropic map $\hat{T}_\varepsilon$ given by the relation from [3] :

Algorithm 1 SINK$(\mathbf{a}, \mathbf{X}, \mathbf{b}, \mathbf{Y}, \varepsilon, \tau, \mathbf{f}_{\text{init}}, \mathbf{g}_{\text{init}})$.

```
f, g ← f_init, g_init (zero by default)
x_1, ..., x_n = X,   y_1, ..., y_m = Y
C ← [h(x_i − y_j)]_ij
while ‖ exp(C − f ⊕ g)/ε) 1_m − a‖_1 < τ do
    f ← ε log a − min_ε(C − f ⊕ g) − f
    g ← ε log b − min_ε(C^⊤ − g ⊕ f) − g
end while
return f, g, P = exp((f ⊕ g − C)/ε)
```

$$\hat{T}_\varepsilon(\mathbf{x}) := \mathbf{x} - \nabla h^* \circ \nabla \hat{\mathbf{f}}_\varepsilon(\mathbf{x}), \tag{6}$$

where $h^*(\mathbf{y}) := \sup_{\mathbf{x}} \langle \mathbf{x}, \mathbf{y}\rangle - h(\mathbf{x})$ is the Fenchel-Legendre transform of the cost function and $\hat{\mathbf{f}}_\varepsilon$ is the empirical approximated potential defined as:

$$\hat{\mathbf{f}}_\varepsilon(\mathbf{x}) := \min_\varepsilon [\mathbf{g}^*_{\varepsilon,j} - h(\mathbf{x} - \mathbf{y}_j)]_j . \tag{7}$$

However those approximations (plan and map) depend heavily on the choice of ε in terms of both accuracy and numerical stabililty (see [4]). For instance, using Sinkhorn algorithm to approximate a map approximation on a large size and spread dataset is not viable : the resulting approximation will be prone to numerical instability as the empirical potential $\hat{\mathbf{f}}_\varepsilon(\mathbf{x})$ is likely to vanish when $\mathbf{x}$ is not close enough to one of the original sample $\mathbf{x}_i$ (see more details in [5]).

[1] when $\varepsilon \longrightarrow 0$, the entropic potentials $(\mathbf{f}_\varepsilon, \mathbf{g}_\varepsilon)$ tend to the optimal potentials $(\mathbf{f}_0, \mathbf{g}_0)$.

FInally, we recall the notion of geodesics in the Wasserstein space in $\mathcal{P}(\mathcal{X})$. For two probability distribution $\mu, \nu \in \mathcal{P}(\mathcal{X})$, the McCann's interpolant between μ and ν is defined as :

$$\mu_\alpha := ((1-\alpha)\mathrm{Id} + \alpha T_0)_\# \mu \,, \tag{8}$$

where $\alpha \in [0,1]$. Conceptually, a geodesic is a *continuous* path between two points of the space (here, two probability distribution).

3 Contribution

3.1 ProgOT, a progressive EOT solver

As the main obstacle for Entropic Optimal Transport appears to be the choice of the regularization term ε which is usually determined using heuristical choice [6] or cross-validation methods [7], machine learning researchers aims to reduce its impact while maintaining a computationally light, stable and accurate algorithm to compute approximation of T_0 and π_0. The principle of ProgOT is to use multiple subroutines of classical EOT solvers (as Sinkhorn's) and iterates across each solution : rather than moving μ to ν with one map, ProgOT iteratively moves μ closer and closer to ν using K entropic maps. At each iteration, it solve the EOT map $T^{(k)}$ between $\mu^{(k)}$, an intermediate distribution, and ν : following the result of $T^{(k)}$, it then computes the next intermediate distribution $\mu^{(k+1)}$. At the end, $\mu^{(K)} = \nu$ and we can compute the final entropic map from μ to ν.

As a warm up, let's consider the first step of the algorithm where $\mu^{(0)} = \mu$. We compute the optimal plan $T^{(0)} = T_0$ from μ to ν. In order to build the intermediate distribution $\mu^{(1)}$, we define the McCann's interpolant $S^{(0)} = (1-\alpha_0)\mathrm{Id} + \alpha_0 T^{(0)}$. By defining $\mu^{(1)} = (S^{(0)})_\# \nu$, we see in Figure 1 that $\mu^{(1)}$ is an intermediate distribution between $\mu^{(0)}$ and ν. If we now compute the optimal plan between $\mu^{(1)}$ and ν, we can recover the overall optimal plan T_0 with $T^{(1)} \circ S^{(0)} = T_0$.

From this simple instance, the algorithm of ProgOT is a straightforward iterative algorithm where at each iteration we move the current distrubtion closer to the target distribution with respect to a coefficient $\alpha_k \in [0,1]$:

$$\mu^{(k+1)} := S^{(k)}_\# \mu^{(k)}, \quad S^{(k)} := (1-\alpha_k)\mathrm{Id} + \alpha_k T^{(k)} \,. \tag{9}$$

Although the algorithm's reasoning is only justified in the ideal setting (no regularization), the authors in [1] shows that the algorithm still converges to the optimal solution when the optimal maps $T^{(k)}$ are actually entropic maps.

Below, we present two algorithms. Algorithm 2 implement ProgOT as a general EOT solver: given the two empirical distributions it returns an optimal map $\mathcal{T}_{\text{prog}}$ and an optimal plan $\mathbf{P}$. Algorithm 3 implements transport map as a result of Sinkhorn potentials solving $(\mathbf{f}^*, \mathbf{g}^*)$: given the Sinkhorn solution, it transport the source point $\mathbf{x}$ to the target population $\mathbf{Y}$.

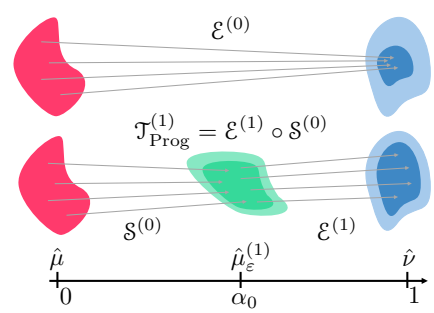


Figure 1: Intuition of 2: By iteratively fitting to the interpolation path, the final transport step is less likely to collapse, resulting in more stable solver.

Algorithm 2 ProgOT$(\mathbf{a}, \mathbf{X}, \mathbf{b}, \mathbf{Y}, (\varepsilon_k, \alpha_k, \tau_k)_k)$

$\mathbf{f} = \mathbf{0}_n, \mathbf{g}^{(-1)} = \mathbf{0}_m$.
for $k = 0, \dots, K$ **do**
  $\mathbf{f}_{\text{init}}, \mathbf{g}_{\text{init}} \leftarrow (1-\alpha_k)\,\mathbf{f}, (1-\alpha_k)\,\mathbf{g}^{(k-1)}$
  $\mathbf{f}, \mathbf{g}^{(k)}, \mathbf{P} \leftarrow \text{Sink}(\mathbf{a}, \mathbf{X}, \mathbf{b}, \mathbf{Y}, \varepsilon_k, \tau_k, \mathbf{f}_{\text{init}}, \mathbf{g}_{\text{init}})$
  $\mathbf{Q} \leftarrow \text{diag}(1/\mathbf{P}\mathbf{1}_m)\mathbf{P}$
  $\mathbf{Z} \leftarrow [\nabla h^*(\sum_j \mathbf{Q}_{ij} \nabla h(\mathbf{x}_i - \mathbf{y}_j))]_i \in \mathbb{R}^{n \times d}$
  $\mathbf{X} \leftarrow \mathbf{X} - \alpha_k \mathbf{Z}$
end for
return: Coupling matrix $\mathbf{P}$,
Map estimator $\mathcal{T}_{\text{prog}}[\mathbf{b}, \mathbf{Y}, (\mathbf{g}^{(k)}, \varepsilon_k, \alpha_k)_k](\cdot)$

Algorithm 3 $\mathcal{T}_{\text{prog}}[\mathbf{b}, \mathbf{Y}, (\mathbf{g}^{(k)}, \varepsilon_k, \alpha_k)_k]$

input: Source point $\mathbf{x} \in \mathbb{R}^d$
initialize: $\mathbf{y} = \mathbf{x}$, α_K reset to 1.
for $k = 0, \dots K$ **do**
  $\mathbf{p} \leftarrow [\mathbf{b}_j \exp(\frac{\mathbf{g}^{(k)} - h(\mathbf{y} - \mathbf{y}_j)}{\varepsilon_k})]_j$
  $\mathbf{p} \leftarrow \mathbf{p}/\mathbf{1}_m^T \mathbf{p} \in \mathbb{R}^m$
  $\mathbf{\Delta} \leftarrow [\nabla h(\mathbf{y} - \mathbf{y}_j)]_j \in \mathbb{R}^{m \times d}$
  $\mathbf{z} \leftarrow \nabla h^*(\mathbf{p}^T \mathbf{\Delta}) \in \mathbb{R}^d$
  $\mathbf{y} \leftarrow \mathbf{y} - \alpha_k \mathbf{z}$.
end for
return: $\mathbf{y}$

3.2 Scheduled Regularization

The authors claim that the main contribution that ProgOT provides is that it reduces the impact of regularization. By progressively moving the source distribution to the target, ProgOT "corrects itself" along each intermediate mapping. This makes ProgOT less prone to inaccuracy by leveraging usual EOT solver's issues such as blurry mapping. Moreover, as ProgOT solves K different EOT problems, each of them can be solved with a different ε_k unveiling a new way to control inaccuracy from regularization.

When solving optimal plan problem, [1] introduces an ε-scheduler able to provide convenient ε_k at each iteration. In the first iterations, ε_k can be loose but as k grows ε_k decreases to an optimal level in order to recover a good approximation of the optimal plan.

To set the regularization parameter $(\varepsilon_k)_k$, the scheduler follows one widely spread heuristical choice which is to start from ε_0 as 5% of the mean of the cost matrix (see [6]). Then, it follows a cross-validation method in order to decide the final regularization parameter ε_K. This cross-validation method finds the best ε_K among $\sigma \times \{s_1, \dots, s_P\}$ where σ is the variance of $\mathbf{Y}$ and $\{s_1, \dots, s_P\}$ are hyperparameters. For each $\varepsilon_p = \sigma s_p$, it computes the optimal entropic map T_p between $\mathbf{Y}$ and itself. The final regularization term ε_K becomes the ε_p which minimizes the following error:

$$\min_p \mathcal{E}_p := ||\mathbf{Y}_{test} - T_p(\mathbf{Y}_{test})||^2 \,, \tag{10}$$

where $\mathbf{Y}_{test}$ is an hold-out evaluation set. The intermediate values are set by interpolating between $\beta_0 \varepsilon_0$ and $\varepsilon_1 = \varepsilon_K$. Here, the only hyperparameters left are β_0 and the scalers terms $(s_p)_p$. Therefore, ProgOT doesn't eliminate the need of tuning hyperparameters but it claims to reduce their impact on the accuracy of its approximation. However, we will see that ProgOT preserves the regularization's sensitivity when it comes to plan approximation.

Algorithm 4 ε-scheduler$(\mathbf{Y}_{\text{test}}, (s_p)_p, \beta_0)$

recover $\mathbf{a}, \mathbf{X}, \mathbf{b}, \mathbf{Y}, (\alpha_k)_k$.
set $\varepsilon_0 \leftarrow \frac{1}{20} \frac{1}{nm} \sum_{ij} h(\mathbf{x}_i - \mathbf{y}_j)$.
set $\sigma \leftarrow \frac{1}{20} \frac{1}{m^2} \sum_{lj} h(\mathbf{y}_l - \mathbf{y}_j)$.
for $p = 1, \dots, P$ **do**
  $\varepsilon \leftarrow s_p \times \sigma$.
  $_, \mathbf{g}, _ \leftarrow \textsc{Sink}(\mathbf{b}, \mathbf{Y}, \mathbf{b}, \mathbf{Y}, \varepsilon, \tau)$
  $T = \mathcal{T}_{\text{prog}}[\mathbf{b}, \mathbf{Y}, (\mathbf{g}, \varepsilon, 1)]$
  $\text{error}_p \leftarrow \|\mathbf{Y}_{\text{test}} - T(\mathbf{Y}_{\text{test}})\|^2$
end for
$p^\star = \text{argmin}_p \text{error}_p$.
$\varepsilon_1 \leftarrow s_{p^\star} \times \sigma$.
$(t_k)_k = (1 - \prod_{\ell=1}^k (1 - \alpha_\ell))_k$.
return: $((1 - t_k)\beta_0 \varepsilon_0 + t_k \varepsilon_1)_k$.

4 Implementation Details

Algorithms 2, 3 and 4 were implemented using only PyTorch primitives. In Section 5, the datasets used are Mix3ToMix10 [8] and Sci-Plex [9]. Mix3ToMix10 can be found at https://github.com/iamalexkorotin/Wasserstein2Benchmark. For [9], raw data containing genes, cells and drug data can be found at https://github.com/cole-trapnell-lab/sci-plex : we include in the implementation a script to process automatically the data following [1] protocol. The implementation alongside with the experiments can be found at https://github.com/a-lagier/ProgOT.

5 Experiments

5.1 Experimental Setup

We run experiments to evaluate ProgOT performances both as a map estimator T_0 and a plan estimator π_0. For each performance, we run ProgOT on synthetic and real data. For synthetic data, we use Mix3ToMix10 dataset from [8]: in this dataset, the source population $\mathbf{X}$ is drawn from a distribution represented by a mixture of 3 Gaussian. The ground-truth T_0 is the gradient of an input convex neural network (ICNN) [10] previously fitted to push $\mathbf{X}$ to a mixture of 10 Gaussians. From this map, we have access to the target population $\mathbf{Y}$. This construction ensures that $\pi_0 = \mathrm{Id}$. For real data, we considered the sci-Plex single cell RNA sequencing data from [9]. It consists of responses of cancers cells to differents drugs reflected by their genes description. As advised by [1], we focus on 5 drugs (Belinostat, Dacinostat, Givinostat, Hesperadin, and Quisinostat). For each drug, we have a population of control (source) cells of size $n \approx 10^4$ and a population of treated (target) cells of size $m \approx 500$. Here we don't have a ground-truth map T_0 nor a plan π_0.

In both parts, we compare ProgOT to Sinkhorn original algorithm. Unless stated otherwise, we use ProgOT with $K = 16$. For map estimator we set $\beta_0 = 5$, $\tau_k \in [0.1, 0.001]$ and $s_p \in \{2^{-3}, \dots, 2^3\}$ for the ε-scheduler.

In those experiments, besides demonstrating that ProgOT outperforms Sinkhorn, we will also perform sensitivity analysis on the hyperparameters of ProgOT as this plays a huge part in EOT algorithm.

5.2 Resources

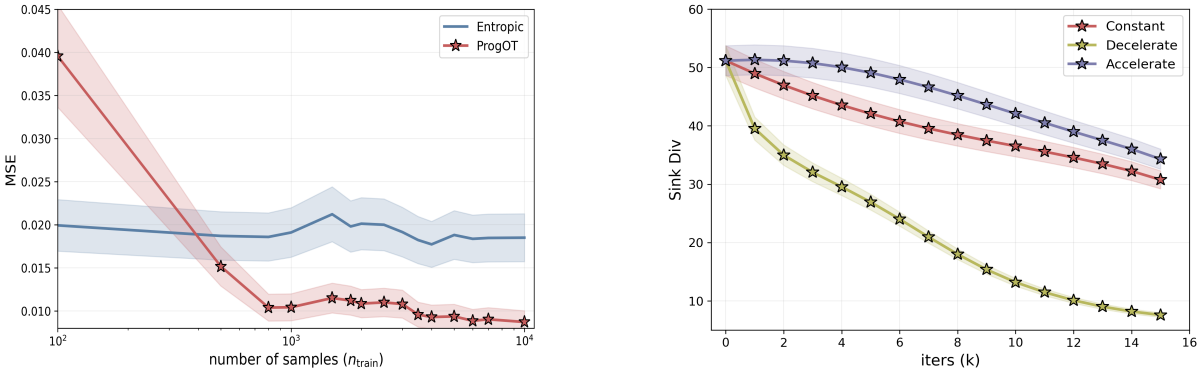
All experiments were conducted on a single Nvidia GA102 [RTX 3090 Ti] GPU with 24GB of memory. All experiments were conducted using Python 3.9 and Torch.

5.3 Experiment 1 : ProgOT as a Map Estimator

First, we demonstrate the convergence of the map computed by ProgOT ($\hat{T}$) and to the true map (T_0), as the number of training points grows. In Figure 2a, we compare ProgOT performances to Sinkhorn using an empirical norm MSE $= \frac{1}{n_{test}} \sum_i ||T_0(x_i) - \hat{T}(x_i)||_2^2$. In opposition to Sinkhorn, ProgOT continue to converge as we increase the number of training samples. In Figure 2b, we visualize the effect of different settings for $(\alpha_k)_k$ and observe their influence over the iterations of K: we compare those performances based on their Sinkhorn divergence $D(\mathbf{X}^{(k)}, \mathbf{Y})$ where $D(\mathbf{X}, \mathbf{Y})$ is defined as,

$$D(\mathbf{X}, \mathbf{Y}) := \mathrm{OT}_{\varepsilon_D}(\mathbf{X}, \mathbf{Y}) - \frac{1}{2}(\mathrm{OT}_{\varepsilon_D}(\mathbf{X}, \mathbf{X}) + \mathrm{OT}_{\varepsilon_D}(\mathbf{Y}, \mathbf{Y}))\,, \tag{11}$$

where $\mathrm{OT}_{\varepsilon_D}$ is the entropic objective value and ε_D is 5% of the mean intra cost of the target population $\mathbf{Y}$. We see that the speed of convergence is directly related to the choice of α_k's. The optimal choice of scheduling $(\alpha_k)_k$ is what the author call "Decelerated" speed, where the source population $\mathbf{X}$ is transported really close to the target at the first steps of the algorithm. As this appears to be a general behavior, we will choose this scheduling for all the other experiments (see more details in Appendix of [1]).



(a) Estimator Consistency of ProgOT, $d = 4$

(b) Efffect of scheduling α_k, $d = 64$

Figure 2: Experiments on Mix3ToMix10 with different dimensions $d = 4, 64$.

We now tackle ProgOT behavior on real data. We compare ProgOT with our baseline on sci-Plex dataset (reduced to top-5 drugs): as cells dimension is originally very large ($d \geq 20,000$), we compare the two methods on different reduced dimensions using PCA. In Table 1, we compare the methods w.r.t the Sinkhorn divergence $D(\hat{T}(\mathbf{X}), \mathbf{Y})$ with $d = 16, 32$. We observe that altough Sinkhorn outputs marginally superior results than ProgOT when $d = 16$, its results get outperformed by ProgOT when $d = 32$.

Method	**Belinostat**		**Dacinostat**		**Givinostat**		**Hesperadin**		**Quisinostat**		
d_{PCA}	16	32	16	32	16	32	16	32	16	32	Drug Rank
ProgOT	14.4	**37.0**	17.1	**34.1**	19.8	**42.0**	**21.8**	**57.1**	15.0	**34.5**	1
Sinkhorn	**13.5**	43.5	**15.0**	50.0	**14.6**	52.4	25.0	89.9	**12.4**	48.5	2

Table 1: Sinkhorn divergence between PROGOT and Sinkhorn methods across different drugs and PCA dimensions.

5.4 Experiment 2 : ProgOT as a Plan Estimator

We now investigate the performances of ProgOT in the task of plan estimator. As pointed by [1], ProgOT does not have a natural ε scheduling when it is used to estimate coupling matrix. For each iteration of ProgOT, we will follow the classical heuristical choice of [6] by putting ε_k as 5% of the mean of the cost matrix between $\mathbf{X}^{(k)}$ and $\mathbf{Y}$. Now, the major parameter that we can tune for ProgOT is the number of iterations: the more iterations we do the more accurate and unblurred we hope the coupling matrix to be.

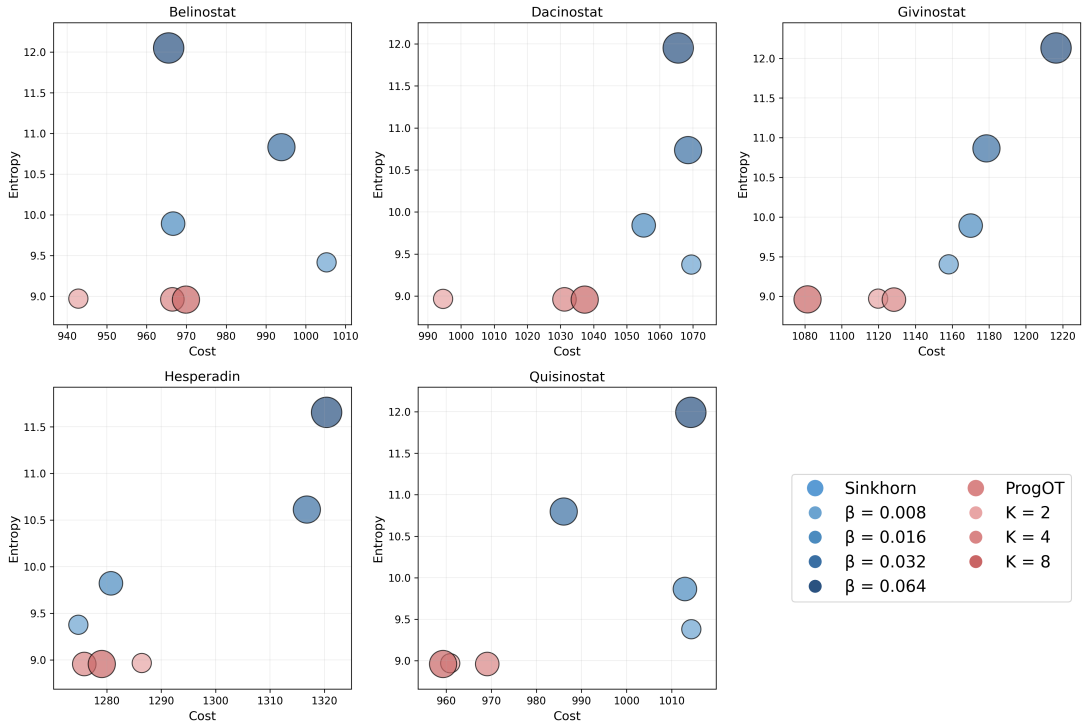


Figure 3: Results of ProgOT and Sinkhorn on the Sci-Plex dataset for different settings. For Sinkhorn, we tune the regularization term $\varepsilon = \beta\varepsilon_0$. For ProgOT, we tune the number of iterations K.

In Figure 3, we display the results of ProgOT and Sinkhorn on the Sci-Plex dataset both with varying parameters: for each solution, we display its entropy and transport cost. From this we can make several observations: (i) ProgOT always obtain the optimal cost on every drug (ii) In ProgOT, increasing K does not consistently make the transport cost decrease (iii) Sinkhorn displays solutions with disparate entropy whereas ProgOT always output solution with uniform entropies. This last observation could constrain the use of ProgOT as some applications may prefer higher entropies for differentiability or better sample complexity.

We now highlight the high dependency on the regularization parameter for ProgOT as a plan estimator. As the ε-scheduler is only designed for map estimation, when we evaluate the plan computed by ProgOT on synthetic data, we recovered the same dependency on ε than the original Sinkhorn algorithm. As shown by Figure 4, we display the progression of the total variation distance d_{TV} between the latent plan π_k and the ground truth optimal plan π_0 we see that the convergence of the plan π_k to π_0 across the iterations highly depends on the choice of ε. When ε is high, the latent plans $(\pi_k)_k$ progressively diverge from the optimal plan π_0. This recreates the need to choose a good regularization term.

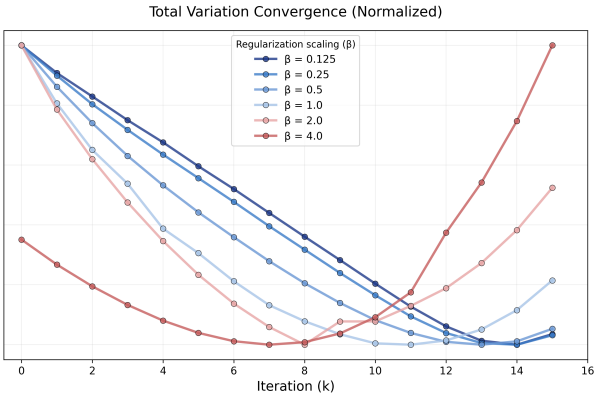


Figure 4: Normalized convergence of ProgOT on the Mix3ToMix10 dataset for regularization settings. Y-axis is normalized and does nto reflect each solution true performance.

6 Conclusion

In this paper, we invetigate the performances and behaviors of ProgOT an Progressive Entropic Optimal Transport solver that uses multiples Sinkhorn subroutines to iteratively solve Kantorovich problem. ProgOT arise as a powerful method to solve OT with better accuracy and provide an entropic method computeing a transport map that convergence to the unregularized transport map. In a practical point of view, it aims to eliminate the tuning of ε that pervade the field of Optimal Transport. As we have seen, this goal is partially satisfied. When computing optimal map, it provides a novel hyperparameters tuning method and stays relatively stable across differents datasets. However, when computing optimal plan, ProgOT inherits the regularization sensitivity of Sinkhorn. Finally, even though ProgOT yields solutions with lower transport cost than Sinkhorn, it is unable to produce solutions with diverse entropies. Further works could try to elaborate ProgOT's iterative method to leverage this latter constraintx.

References

[1] Parnian Kassraie, Aram-Alexandre Pooladian, Michal Klein, James Thornton, Jonathan Niles-Weed, and marco cuturi. Progressive entropic optimal transport solvers. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.

[2] Yann Brenier. Polar factorization and monotone rearrangement of vector-valued functions. *Communications on Pure and Applied Mathematics*, 44(4):375–417, 1991.

[3] Aram-Alexandre Pooladian and Jonathan Niles-Weed. Entropic estimation of optimal transport maps, 2024.

[4] Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transportation distances, 2013.

[5] Aude Genevay, Lénaic Chizat, Francis Bach, Marco Cuturi, and Gabriel Peyré. Sample complexity of sinkhorn divergences, 2019.

[6] Marco Cuturi, Laetitia Meng-Papaxanthos, Yingtao Tian, Charlotte Bunne, Geoff Davis, and Olivier Teboul. Optimal transport tools (ott): A jax toolbox for all things wasserstein, 2022.

[7] Adrien Vacher and François-Xavier Vialard. Parameter tuning and model selection in optimal transport with semi-dual brenier formulation. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS '22, Red Hook, NY, USA, 2022. Curran Associates Inc.

[8] Alexander Korotin, Lingxiao Li, Aude Genevay, Justin M Solomon, Alexander Filippov, and Evgeny Burnaev. Do neural optimal transport solvers work? a continuous wasserstein-2 benchmark. *Advances in Neural Information Processing Systems*, 34, 2021.

[9] Sanjay R. Srivatsan, José L. McFaline-Figueroa, Vijay Ramani, Lauren Saunders, Junyue Cao, Jonathan Packer, Hannah A. Pliner, Dana L. Jackson, Riza M. Daza, Lena Christiansen, Fan Zhang, Frank Steemers, Jay Shendure, and Cole Trapnell. Massively multiplex chemical transcriptomics at single-cell resolution. *Science*, 367(6473):45–51, 2020.

[10] Brandon Amos, Lei Xu, and J. Zico Kolter. Input convex neural networks, 2017.